

TransGrokking

理论框架与实验规范

固定规范版



实验平台: NVIDIA GeForce RTX 4060 Laptop GPU 8GB

Conda 环境: 项目根目录 `./env`

基础任务: $\mathbb{Z}_{97}$ 上的模加法

文档性质: 理论定义、实验协议与分析规范

摘要

TransGrokking 以有限循环群上的模加法为实验台, 研究小型 Transformer 在训练集拟合之后出现的延迟泛化动力学。本文建立一套长期稳定的理论框架与实验规范, 统一描述模型行为、函数结构、内部表征、计算电路和优化动力学。

理论主轴包括完整 logit 张量、群对称性、Reynolds 投影、三维 Fourier 支撑、算法 margin、非等变干扰和权重衰减下的回路竞争。实验主轴采用密集 checkpoint、完整函数重建、跨层结构测量和 checkpoint 分支干预, 用于定位训练集记忆、算法结构形成、残差清理和测试准确率跃迁之间的时间关系。

文档性质

本文作为固定指导文档使用。正文规定任务、符号、指标、实验条件、数据产物和判定逻辑。运行记录、工程日志与阶段性结果由独立文件维护。

目录

摘要	i
第一章 项目定位与固定约束	1
1.1 研究使命	1
1.2 科学证据原则	1
1.3 固定平台与环境	1
1.4 参考 CE-only 配置	2
1.5 软件职责边界	2
第二章 理论框架	4
2.1 完整 logits 与中心化	4
2.2 行为事件与可见阶段	4
2.3 压缩视角	5
2.4 群作用与 Reynolds 投影	5
2.5 有限 Fourier 空间	6
2.6 算法 margin 与残差干扰	6
2.7 权重衰减的作用	7
2.8 Congruence loss	7
2.9 训练阶段假说	8
第三章 实验总体设计	9
3.1 五层证据体系	9
3.2 checkpoint 时间轴	9
3.3 运行产物协议	9
3.4 RTX 4060 Laptop 8GB 资源设计	10
3.5 运行矩阵的分阶段展开	11
第四章 实验协议	12
4.1 协议一：CE-only 基准轨迹	12

4.1.1	执行步骤	12
4.1.2	停止规则	12
4.1.3	验收	12
4.2	协议二：函数空间分析	12
4.2.1	实现对象	13
4.2.2	数学测试	13
4.2.3	第一张机制图	13
4.3	协议三：Fourier 分析	13
4.4	协议四：表征与电路分析	13
4.4.1	Embedding	14
4.4.2	Hidden state	14
4.4.3	电路因果测试	14
4.5	协议五：优化动力学与分支干预	14
4.6	协议六：Congruence 成对实验	14
4.7	协议七：统一分析报告	14
第五章	核心假设与判定逻辑	16
5.1	解释边界	16
第六章	执行规范	18
6.1	推荐实施顺序	18
6.2	基准运行命令模板	18
6.3	单次运行记录规范	19
附录 A	指标速查	20
附录 B	参考资料	21

第 1 章

项目定位与固定约束

1.1 研究使命

基础任务为

$$a, b \in \mathbb{Z}_p, \quad y = (a + b) \bmod p. \quad (1.1)$$

项目把每个 checkpoint 视为一个独立模型状态，测量可见行为与内部结构的连续变化。研究目标是构建一条具备精确恢复、离线重算和因果分支能力的训练时间线，并对 Grokking 期间的函数重组过程形成可检验解释。

1.2 科学证据原则

- 训练曲线负责定位事件，机制结论依赖函数指标、表征指标和干预证据。
- 所有正式结果绑定代码版本、运行标识、配置、数据划分和 checkpoint step。
- 理论假说与经验观察分开记录，任何假说均允许被单独否定。
- 失败或中断运行保留配置、状态和错误摘要，用于判断执行边界。

1.3 固定平台与环境

正式实验固定使用 `cuda:0` 上的 NVIDIA GeForce RTX 4060 Laptop GPU 8GB。项目只使用根目录 Conda prefix:

```
conda run --prefix ./env python -m transgrokking.cli doctor
conda run --prefix ./env python -m transgrokking.cli doctor \
  --require-cuda \
  --expected-device "NVIDIA GeForce RTX 4060 Laptop GPU" \
  --expected-vram-gb 8
```

主基线采用 FP32，关闭 TF32、AMP、BF16、FP16 和 `torch.compile`。每个正式 run 保存 Python、PyTorch、CUDA runtime、设备名称、compute capability、峰值 allocated VRAM 和峰值 reserved VRAM。

1.4 参考 CE-only 配置

表 1.1: CE-only 基准条件

类别	字段	规定值
任务	modulus / train fraction / split seed	97 / 0.4 / 42
模型	width / heads / layers / MLP	128 / 4 / 2 / 512
模型	dropout / norm-first / final norm	0 / true / false
优化	optimizer / lr / weight decay	AdamW / 10^{-3} / 0.5
参数策略	decay	matrix weights 与 embeddings
参数策略	no decay	bias 与 LayerNorm
训练	initial max steps / seed / device	5000 / 1 / cuda:0
数值	precision / TF32 / AMP	FP32 / false / false
记录	evaluation / checkpoint	50 / 100 step
损失	CE / congruence	1.0 / 0.0

1.5 软件职责边界

```

TransGrokking/
|-- AGENTS.md
|-- README.md
|-- environment.yml
|-- pyproject.toml
|-- configs/
|-- docs/
|-- src/transgrokking/
| |-- models/
| |-- training/
| |-- metrics/
| |-- interventions/
| `-- utils/

```

```
|-- tests/  
|-- analysis/  
|-- legacy/  
`-- runs/
```

训练循环、数学指标、分析绘图和干预逻辑维持清晰边界。配置文件承担实验参数入口，运行目录承担科学证据载体。

第 2 章

理论框架

2.1 完整 logits 与中心化

模型对输入 (a, b) 产生 p 个候选输出分数：

$$z_\theta(a, b) = (z_\theta(a, b, 0), \dots, z_\theta(a, b, p-1)). \quad (2.1)$$

softmax 概率与预测为

$$P_\theta(c | a, b) = \frac{e^{z_\theta(a, b, c)}}{\sum_j e^{z_\theta(a, b, j)}}, \quad \hat{y}_\theta(a, b) = \arg \max_c z_\theta(a, b, c). \quad (2.2)$$

所有输入与候选输出组成三维函数

$$z_\theta : \mathbb{Z}_p^3 \rightarrow \mathbb{R}. \quad (2.3)$$

softmax 对类别公共平移保持不变，因此函数分析使用中心化 logits：

$$\tilde{z}(a, b, c) = z(a, b, c) - \frac{1}{p} \sum_{j=0}^{p-1} z(a, b, j). \quad (2.4)$$

2.2 行为事件与可见阶段

单样本分类 margin 定义为

$$m(a, b) = z_y(a, b) - \max_{c \neq y} z_c(a, b). \quad (2.5)$$

行为时间线记录 train/test margin 的均值、最小值和固定分位数。事件检测使用连续 evaluation 窗口：

$$t_{\text{fit}} = \min\{t : \text{Acc}_{\text{tr}}(t) \geq 0.999\}, \quad (2.6)$$

$$t_{\text{grok50}} = \min \left\{ t : \text{Acc}_{\text{te}}(t) \geq \frac{1 + 1/p}{2} \right\}, \quad (2.7)$$

$$t_{\text{grok99}} = \min\{t : \text{Acc}_{\text{te}}(t) \geq 0.99\}. \quad (2.8)$$

这些事件定位训练曲线中的可见阶段，函数空间指标用于描述内部结构变化。

2.3 压缩视角

项目采用任务对齐压缩命题：在保持训练似然与分类 margin 的条件下，模型逐渐消除与任务对称性无关的信息，并形成跨样本共享的函数结构。

对给定参数化定义最小实现代价

$$\mathcal{C}(z) = \inf_{\theta: f_{\theta}=z} \|\theta\|_2^2. \quad (2.9)$$

若算法型函数 $z_{\mathcal{A}}$ 的实现代价低于样本记忆型函数 $z_{\mathcal{M}}$ ，权重衰减会在长期动力学中提高前者的相对优势。参数范数受 LayerNorm 和重参数化影响，因此该命题需要函数指标共同检验。

2.4 群作用与 Reynolds 投影

模加法关系在群作用

$$T_{u,v}(a, b, c) = (a + u, b + v, c + u + v) \quad (2.10)$$

下保持不变量

$$d = c - a - b \pmod{p}. \quad (2.11)$$

沿群轨道平均定义 Reynolds 投影：

$$(\Pi z)(a, b, c) = \frac{1}{p^2} \sum_{u, v \in \mathbb{Z}_p} z(a + u, b + v, c + u + v). \quad (2.12)$$

投影结果只依赖 d 。实际计算可直接构造 offset profile

$$g(d) = \frac{1}{p^2} \sum_{a, b \in \mathbb{Z}_p} \tilde{z}(a, b, a + b + d). \quad (2.13)$$

由此得到

$$z^{\parallel}(a, b, c) = g(c - a - b), \quad z^{\perp} = (I - \Pi)\tilde{z}. \quad (2.14)$$

Reynolds 算子满足 $\Pi^2 = \Pi$ ，并在均匀内积下给出正交分解

$$\tilde{z} = z^{\parallel} + z^{\perp}, \quad \langle z^{\parallel}, z^{\perp} \rangle = 0. \quad (2.15)$$

非等变能量比例定义为

$$D_{\text{eq}} = \frac{\|z^{\perp}\|_2^2}{\|\tilde{z}\|_2^2}. \quad (2.16)$$

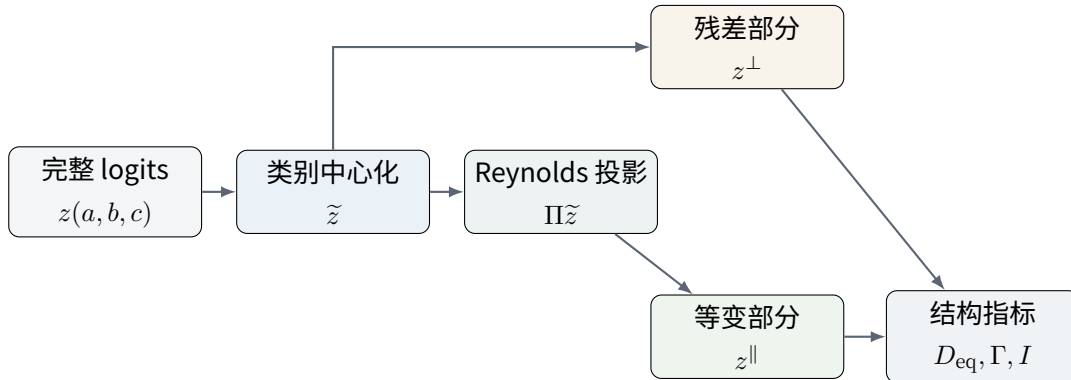


图 2.1: 完整 logits 到函数空间指标的分析管线

2.5 有限 Fourier 空间

令

$$\omega = e^{2\pi i/p}, \quad \chi_r(x) = \omega^{rx}. \quad (2.17)$$

三维 Fourier 展开为

$$\tilde{z}(a, b, c) = \sum_{r_a, r_b, r_c} \hat{z}(r_a, r_b, r_c) \omega^{r_a a + r_b b + r_c c}. \quad (2.18)$$

模加法目标张量满足

$$Y(a, b, c) = \mathbf{1}[c = a + b \pmod{p}] = \frac{1}{p} \sum_{r=0}^{p-1} \omega^{r(a+b-c)}. \quad (2.19)$$

因此目标 Fourier 支撑集中于

$$\mathcal{L} = \{(r, r, -r) : r \in \mathbb{Z}_p\}. \quad (2.20)$$

对任意单一模式执行群平均，会出现两个有限几何和。模式保留条件为

$$r_a + r_c = 0, \quad r_b + r_c = 0, \quad (2.21)$$

从而得到 $r_a = r_b = -r_c$ 。目标线能量比例定义为

$$E_{\text{line}} = \frac{\sum_r |\hat{z}(r, r, -r)|^2}{\sum_{r_a, r_b, r_c} |\hat{z}(r_a, r_b, r_c)|^2}. \quad (2.22)$$

统一采用正交归一化时，Parseval 定理给出

$$E_{\text{line}} = 1 - D_{\text{eq}}. \quad (2.23)$$

式 (2.23) 同时承担数学解释和实现校验作用。

2.6 算法 margin 与残差干扰

算法投影只依赖 offset profile，定义全局算法 margin

$$\Gamma = g(0) - \max_{d \neq 0} g(d). \quad (2.24)$$

定义非等变残差的最大对抗干扰

$$I = \max_{a, b, c \neq y} [z^\perp(a, b, c) - z^\perp(a, b, y)]. \quad (2.25)$$

对任意错误类别 $c \neq y$ ，有

$$\tilde{z}(a, b, y) - \tilde{z}(a, b, c) = [z^\parallel(a, b, y) - z^\parallel(a, b, c)] + [z^\perp(a, b, y) - z^\perp(a, b, c)] \quad (2.26)$$

$$\geq \Gamma - I. \quad (2.27)$$

因此得到充分条件

$$\boxed{\Gamma > I \implies \hat{y}(a, b) = a + b \pmod{p} \text{ 对全部输入成立.}} \quad (2.28)$$

定义

$$t_{\text{alg}} = \min\{t : \Gamma_t > 0\}, \quad t_{\text{dom}} = \min\{t : \Gamma_t > I_t\}. \quad (2.29)$$

若 $t_{\text{alg}} < t_{\text{grok99}}$ 且 t_{dom} 与 t_{grok99} 接近, 实验将支持“算法结构提前形成, 残差清理触发可见跃迁”的过程解释。

2.7 权重衰减的作用

带平方范数惩罚的连续梯度流为

$$\dot{\theta} = -\nabla_{\theta} L_{\text{data}} - \lambda \theta. \quad (2.30)$$

AdamW 的简化离散更新为

$$\theta_{t+1} = (1 - \eta_t \lambda) \theta_t - \eta_t P_t \nabla_{\theta} L_{\text{data}}. \quad (2.31)$$

权重衰减对 logits 的作用经 Jacobian $J_{\theta} = \partial z / \partial \theta$ 传递。两回路近似写为

$$z_t = a_t z_{\mathcal{A}} + b_t z_{\mathcal{M}}, \quad (2.32)$$

并用有效代价

$$\mathcal{R}(a, b) = \frac{\lambda}{2} (c_{\mathcal{A}} a^2 + c_{\mathcal{M}} b^2) \quad (2.33)$$

描述算法回路和记忆回路的范数压力。若 $c_{\mathcal{M}} > c_{\mathcal{A}}$, 记忆回路在训练后期承担更强收缩。假设干扰近似指数下降:

$$I_t \approx I_{t_0} e^{-\lambda_{\text{eff}}(t-t_0)}, \quad (2.34)$$

并且算法 margin 接近 $\Gamma_{\infty} > 0$, 则

$$t_{\text{dom}} - t_0 \approx \frac{1}{\lambda_{\text{eff}}} \log \frac{I_{t_0}}{\Gamma_{\infty}}. \quad (2.35)$$

式 (2.35) 给出可检验的延迟律。其可信度依赖参数更新分解、 D_{eq} 、 Γ 和 I 的联合证据。

2.8 Congruence loss

圆周距离惩罚为

$$L_{\text{cong}} = \sum_{k=0}^{p-1} P_{\theta}(k | a, b) \left[1 - \cos \frac{2\pi(k-y)}{p} \right]. \quad (2.36)$$

令 $\omega = e^{2\pi i/p}$, 可改写为

$$L_{\text{cong}} = 1 - \text{Re} \left[\omega^{-y} \sum_k P_{\theta}(k | a, b) \omega^k \right]. \quad (2.37)$$

该项直接监督预测分布的第一圆周 Fourier 矩。成对实验需要检查 t_{alg} 、 t_{dom} 、各目标频率能量和最终函数结构。

2.9 训练阶段假说

表 2.1: 待检验的训练阶段模板

阶段	行为特征	函数与表示特征
Memorization	train accuracy 快速饱和, test 接近随机	D_{eq} 高, 训练 residual margin 增长, 频谱较分散
Circuit formation	test 仍可能较低	Γ 增长, $L_{ }$ 下降, 目标线能量提高
Cleanup	test 进入快速改善区间	I 与 D_{eq} 下降, 非目标频率衰减
Stable regime	train/test 均处于高位	频谱、电路归因和算法 margin 趋稳

该模板承担分析组织作用。实际轨迹可以出现阶段重叠、顺序变化或缺失。

第 3 章

实验总体设计

3.1 五层证据体系

表 3.1: 全过程观测层级

层级	核心对象	主要指标与工具
行为	train/test 前向结果	CE、accuracy、margin 分位数、错误 offset、事件时间
函数	完整 $z(a, b, c)$	D_{eq} 、 E_{line} 、 Γ 、 I 、 $L_{ }$
表征	embedding 与 hidden state	Fourier 能量、circle fit、effective rank、linear probe
电路	attention、MLP、unembedding	attribution、ablation、patching、module transplant
优化	参数、梯度、AdamW update	范数、data/decay 分解、径向/切向分量、分支干预

3.2 checkpoint 时间轴

基准轨迹采用 evaluation interval 50 和 checkpoint interval 100。二者属于执行配置，不改变任务、模型、优化和数值条件。

关键 checkpoint 覆盖初始化、拟合前、 t_{fit} 、 t_{alg} 、测试跃迁前、 t_{dom} 、 t_{grok50} 、 t_{grok99} 和后期稳定状态。事件检测完成后，从相邻 checkpoint 中选取代表状态进行深度分析。

3.3 运行产物协议

```
runs/<run_id>/
|-- config.resolved.yaml
```

```

|-- metadata.json
|-- split.pt
|-- status.json
|-- metrics/
| |-- scalars.jsonl
| |-- error_offsets.jsonl
| `-- events.json
|-- checkpoints/
| |-- step_000000.pt
| `-- manifest.json
|-- tensors/
|-- figures/
`-- logs/

```

checkpoint 包含模型、优化器、global step、配置、split hash 和 Python/NumPy/Torch RNG 状态。child run 记录父 run、父 checkpoint 和父 step。

3.4 RTX 4060 Laptop 8GB 资源设计

对于 $p = 97$:

表 3.2: 典型 FP32 张量的近似内存规模

张量	近似大小
完整 logits [97, 97, 97]	3.48 MiB
单层 residual [9409, 2, 128]	9.19 MiB
单层 MLP activation [9409, 2, 512]	36.75 MiB
两层四头 attention pattern [9409, 2, 4, 2, 2]	1.15 MiB

完整 logits 可逐 checkpoint 生成。activation 只在指定模块和指定 checkpoint 提取，随后立即转移到 CPU。Fourier、Reynolds 和绘图默认允许在 CPU 侧执行。

3.5 运行矩阵的分阶段展开

表 3.3: 建议运行矩阵

实验组	条件	seed	目的
CE-reference	CE, WD=0.5	1	建立代表性基准轨迹
CE-replication	CE, WD=0.5	2,3	检查事件顺序与指标关系的稳定性
WD-grid	CE, WD=0 / 0.1 / 1.0	1-3	分析权重衰减对延迟和 margin 的影响
Congruence-paired	CE + congruence 0.5	与 CE 配对	检查低阶循环结构的学习动力学
Loss-schedule	late-on / early-only	代表 seed	定位 congruence loss 的作用阶段
Checkpoint-branch	参数与模块干预	代表轨迹	获取因果证据

广泛超参数扫描安排在函数空间与 Fourier 分析管线稳定之后，使每条长程轨迹都具有充分的结构测量。

第 4 章

实验协议

4.1 协议一：CE-only 基准轨迹

4.1.1 执行步骤

1. 确认代码版本、`./env` 与配置文件；运行普通 doctor 和严格 doctor。
2. 复制 resolved config, 固定 split hash 与 scientific config hash。
3. 使用 seed 1 运行到 5000 step, 保存全部状态和峰值显存。
4. 若 t_{grok99} 未出现, 保持 scientific config, 恢复至 20000 step; 仍未出现时恢复至 50000 step。
5. 运行离线 evaluator, 核对 timeline、manifest 与 checkpoint 一致性。

4.1.2 停止规则

正式轨迹在以下条件之一满足时结束：检测到 t_{grok99} 后继续训练至少 20 个 evaluation interval；达到 50000 step；训练进入 failed/interrupted 状态。failed/interrupted run 保留完整错误与状态证据。

4.1.3 验收

- run 状态、metadata、split、metrics 和 checkpoint manifest 完整；
- scalar step 严格递增；
- 恢复后 scientific hash 与 split hash 保持一致；
- 正式平台 metadata 匹配 RTX 4060 Laptop 8GB；
- 实际结果的描述限制在对应 run 支持范围内。

4.2 协议二：函数空间分析

4.2.1 实现对象

实现完整 logits evaluator 和纯函数指标: 式 (2.4)、式 (2.13)、式 (2.16)、式 (2.24) 与式 (2.25)。

每个分析 checkpoint 保存 offset profile 和聚合指标。完整 logits 依据配置选择持久化；关键 checkpoint 可以保留张量，其余状态通过离线前向重建。

4.2.2 数学测试

测试	条件
中心化	每个 (a, b) 上 $\sum_c \tilde{z}(a, b, c) \approx 0$
幂等性	$\Pi^2 z \approx \Pi z$
正交性	$\langle z^{\parallel}, z^{\perp} \rangle \approx 0$
重构	$\tilde{z} \approx z^{\parallel} + z^{\perp}$
群不变性	$z^{\parallel}(T_{u,v}(a, b, c)) = z^{\parallel}(a, b, c)$
阈值案例	人工构造 $\Gamma > I$ 时全表预测正确

4.2.3 第一张机制图

把 Γ_t 、 I_t 、train/test accuracy 和 $D_{\text{eq}}(t)$ 放在共享时间轴。该图优先回答：算法结构何时可用，残差何时失去推翻能力，测试跃迁何时发生。

4.3 协议三：Fourier 分析

统一使用

```
torch.fft.fftn(centered_logits, dim=(0, 1, 2), norm="ortho")
```

实现目标线 mask、各频率能量和 inverse transform。验收必须包含 Parseval 和式 (2.23) 的数值一致性。

在最终 grokked checkpoint 中选取解释目标线 95% 能量的最小频率集合 F^* 。随后固定 F^* ，回看整个时间线，构建 restricted logits 与 excluded logits。这样可以观察结构回路形成和记忆成分清理的连续进程。

4.4 协议四：表征与电路分析

4.4.1 Embedding

沿 token 轴计算

$$\hat{E}(r, :) = \frac{1}{p} \sum_x E(x, :) e^{-2\pi i r x / p}. \quad (4.1)$$

对每个频率拟合正弦与余弦方向，并记录解释方差。PCA 图承担直观辅助作用，Fourier fit 提供任务对齐证据。

4.4.2 Hidden state

对 $H_l(a, b, :)$ 在输入平面执行二维 DFT，记录关键 (r, r) 模式。冻结 checkpoint 表征后训练线性 probe，用 held-out pair 检查层间算法信息。

4.4.3 电路因果测试

候选 head、MLP neuron 和 unembedding 方向先通过 attribution 定位。后续使用频率消融、activation patching、模块冻结和 early/late 模块移植检验其功能。

4.5 协议五：优化动力学与分支干预

对实际 AdamW parameter groups 记录

$$\Delta\theta_{\text{decay}} = -\eta\lambda\theta, \quad R_{\text{decay/data}} = \frac{\|\Delta\theta_{\text{decay}}\|}{\|\Delta\theta_{\text{data}}\|}. \quad (4.2)$$

将 data update 分解为参数方向上的径向分量和正交的切向分量。基于同一 checkpoint 建立 WD-off、WD-low、WD-high、optimizer-reset 和 module-freeze 分支。

分支结果使用绝对 global step 和相对 branch step 双坐标展示。每个分支记录父 run、父 checkpoint、修改项与 scientific config 差异。

4.6 协议六：Congruence 成对实验

配对运行共享数据划分、模型初始化和数值配置。四个核心条件为 CE、全程 congruence、 t_{fit} 后开启、 t_{fit} 后关闭。

额外记录 CE 梯度与 congruence 梯度在各模块上的范数和夹角。若低阶目标频率在训练早期提前增长，主要证据指向学习动力学改变；若最终频谱和 margin 稳定改变，主要证据指向函数结构偏置改变。

4.7 协议七：统一分析报告

报告脚本只读取已有 run artifacts。每个代表 run 生成：

- 行为时间线与事件表;
- D_{eq} 、 Γ 、 I 和 $L_{\parallel}$;
- frequency-time heatmap 与 restricted/excluded loss;
- embedding/hidden-state 结构指标;
- 模块范数与 update 分解;
- checkpoint 分支对比和配置摘要。

报告中的观察与机制解释分开书写。每条解释附带对应指标、图表和干预证据。

第 5 章

核心假设与判定逻辑

H1: 算法结构提前形成

若 $t_{\text{alg}} < t_{\text{grok99}}$, 并且 $L_{\parallel}$ 在测试跃迁以前持续下降, 算法结构可能已经存在。restricted model 或 key-frequency projection 应复现部分泛化能力。

H2: 阈值穿越解释准确率跃迁

若 $\Gamma_t - I_t$ 由负转正的时间邻近 test accuracy 快速上升区间, 式 (2.28) 提供直接解释。人工缩放 residual 可以进一步检验阈值的因果作用。

H3: cleanup 对应非等变残差清理

若训练后期 D_{eq}, I 与非目标 Fourier 能量同步下降, cleanup 具有函数空间证据。checkpoint 后关闭 weight decay 可以检验清理速度的依赖关系。

H4: 算法回路具有更高的 margin-norm 效率

记录 $\Gamma/\|\theta\|_2$ 和模块级对应量。若算法阶段该量持续上升, 并在 WD 分支中表现稳定, 参数复杂度解释获得支持。

H5: congruence 优先增强低阶循环模式

成对初始化下, 比较 $e_r(t) = |\hat{z}(r, r, -r)|^2$ 、 t_{alg} 和梯度夹角。late-on 与 early-only 调度用于定位作用阶段。

5.1 解释边界

- $z^{\perp}$ 精确描述对称性破坏, 仍可能遗漏等变子空间内部的复杂过拟合。
- Fourier 稀疏提供结构证据, 电路因果性需要消融、投影和 patching。
- 参数范数受架构和归一化影响, 跨模型比较优先使用函数指标和归一化 margin。
- 完整运算表可用于机制观测, 超参数选择需要预先固定规则。

- 精度、TF32、dropout 与随机性设置都可能改变 Grokking 时间。

第 6 章

执行规范

6.1 推荐实施顺序

表 6.1: 固定实施顺序与验收对象

顺序	工作	验收对象
1	CE-only 基准轨迹	长程 run、完整行为时间线、安全恢复与硬件 metadata
2	函数空间分析	Reynolds 数学测试、 $\Gamma/I/D_{eq}$ 时间线与阈值图
3	Fourier 分析	Parseval、频域投影等价、频率热图与 restricted/excluded 指标
4	多 seed 与 WD 网络	事件关系稳定性、延迟分布和范数效率比较
5	表征、电路与 checkpoint 分支	模块归因、消融、patching 和分支因果结果
6	Congruence 成对条件	梯度关系、频率增长、作用阶段和最终结构比较

6.2 基准运行命令模板

```
conda run --prefix ./env python -m transgrokking.cli doctor

conda run --prefix ./env python -m transgrokking.cli doctor \
  --require-cuda \
  --expected-device "NVIDIA GeForce RTX 4060 Laptop GPU" \
  --expected-vram-gb 8

conda run --prefix ./env python -m transgrokking.cli train \
  --config configs/baseline_ce.yaml
```

若 5000 step 内未检测到预定事件, 使用已有 checkpoint 提高 execution config 中的 max_steps,

并保持 scientific config hash 一致。

6.3 单次运行记录规范

每个正式 run 的记录至少包括：

- 代码版本、resolved config、scientific config hash 和 split hash；
- 设备、软件环境、随机种子、parameter groups 和数值精度；
- 生命周期状态、峰值显存、checkpoint manifest 和恢复来源；
- 行为事件、函数指标、分析 checkpoint 和派生图表；
- 异常、重试、执行配置调整和解释边界。

第 A 章

指标速查

符号	定义
$\tilde{z}$	按候选类别中心化的完整 logits
$g(d)$	Reynolds 投影对应的一维 offset profile
$z^{\parallel}$	满足模加法平移对称性的 logits 成分
$z^{\perp}$	非等变残差 logits
D_{eq}	非等变能量比例
E_{line}	三维 Fourier 目标线能量比例
Γ	算法投影的全局 margin
I	非等变残差的最大分类干扰
t_{fit}	训练准确率达到阈值并持续满足的首次窗口起点
t_{alg}	Γ_t 首次为正的时间
t_{dom}	$\Gamma_t > I_t$ 首次成立的时间
t_{grok99}	测试准确率达到 99% 并持续满足的首次窗口起点

第 B 章

参考资料

参考文献

- [1] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. arXiv:2201.02177, 2022.
- [2] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress Measures for Grokking via Mechanistic Interpretability. arXiv:2301.05217, 2023.
- [3] Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking Beyond Algorithmic Data. arXiv:2210.01117, 2022.